

Predecir la deserción escolar y el éxito académico

Inteligencia Artificial

Profesor: Manuel Alejandro Goyo

Integrantes:

- Nicolas Arellano
- Miguel Cornejo
- Thomas Riveros

10 de Noviembre 2025



Índice

- 1 Introducción y Contexto del Problema
- 2 Metodología de Evaluación
- 3 Desarrollo de la Predicción
- 4 Resultados
- 5 Conclusiones
- 6 Bibliografía



1. Introducción y Contexto del Problema

El Desafío de la Deserción en Educación Superior

La deserción estudiantil representa uno de los retos más significativos en el ámbito de la educación superior a nivel global.

Este problema es la capacidad de identificar tempranamente a los estudiantes en riesgo de abandonar sus estudios permitiría implementar estrategias de intervención proactivas y personalizadas.

¿Por Qué se preocupa la deserción Universitaria?

El contexto real:

- Miles de estudiantes abandonan sus estudios cada año
- Las instituciones invierten recursos sin saber qué estudiantes están en riesgo
- No existe un sistema de alerta temprana efectivo





1. Introducción y Contexto del Problema

Objetivo General

Se enfoca en desarrollar un modelo de machine learning capaz de predecir el destino académico de los estudiantes universitarios, clasificándolos en tres categorías: aquellos que probablemente abandonarán sus estudios, que permanecerán matriculados pero sin graduarse y aquellos que lograrán graduarse exitosamente.

La predicción se realiza utilizando información disponible desde el momento de la inscripción y durante los primeros semestres académicos.





1. Introducción y Contexto del Problema

Contexto del Dataset

Este dataset proviene de una institución de educación superior en Portugal, recopilado en un proyecto financiado por el programa SATDAP. Cuenta con **dos archivos** principales: **train(1).csv** para el entrenamiento del modelo, que incluye la variable objetivo, y **test(2).csv** para la evaluación, que carece de esta variable objetivo. Comparten la misma estructura de **36 variables** predictoras que capturan diferentes dimensiones del perfil estudiantil.

Distribución desbalanceada:

- 47% Graduate
- 32% Dropout
- 21% a Enrolled

Categorización de las Variables al momento de Inscripción

Demografía y Personal

- Edad, género y nacionalidad
- Estado civil y situación familiar
- Condiciones de discapacidad

Socioeconómicas

- Tipo de escuela secundaria
- Método de admisión utilizado
- Becas y situación de deudas
- Tipo de alojamiento durante estudios

Macroeconómico

- Tasa de desempleo
- Tasa de inflación
- Producto Interno Bruto

Académicas Previas

- Notas de educación secundaria
- Calificaciones de admisión
- Tipo de institución de origen

Rendimiento Académico Inicial

- Calificaciones primeros dos semestres
- Créditos aprobados inicialmente
- Semestres de inscripción cursados

Variable Objetivo

- **categorica con tres clases (Dropout, Enrolled, Graduate)**



2. Metodología de Evaluación

Evaluación

La evaluación de las sumisiones se basa en el **Accuracy**, una métrica simple y directa para clasificación multiclase que mide la proporción de predicciones correctas respecto al total de muestras.

Esta métrica es intuitiva y fácil de interpretar, se cuenta el desbalanceo de clases en el dataset (mayoría en "Graduate"), por lo que un modelo que prediga siempre la clase mayoritaria podría obtener un score alto (~0.47) sin ser realmente efectivo. Recomendando validar con F1-macro localmente para un mejor balance.

Fórmula

$$\text{Accuracy} = \frac{\text{número de predicciones correctas}}{\text{número total de muestras}}$$

Donde una predicción es correcta si la clase predicha (determinada por el valor máximo en las columnas de one-hot) coincide con la clase verdadera.

Umbral de éxito

Baseline simple ~0.47 (mayoritario);
apunta a >0.70 para top.

Resultado esperado de salida

submission.csv (14.08 kB)

id	Dropout	Enrolled	Graduate
0	0.0	0.0	1.0
1	0.0	0.0	1.0
2	0.0	0.0	1.0
3	0.0	0.0	1.0
4	0.0	0.0	1.0
5	0.0	0.0	1.0



3. Desarrollo de la Predicción

1. Carga

Train: /kaggle/input/predecir-la-desercion-escolar-y-el-exito-academico/train (2).csv

Test: /kaggle/input/predecir-la-desercion-escolar-y-el-exito-academico/test (1).csv

- Cargar Train (X_train, y_train) y Test
- Codificación de Target (LabelEncoder)
- Estandarización de Features (StandardScaler)

Dataset de Entrenamiento

Filas:	3,539
Columnas:	37
Features:	36
Target:	1 (3 clases)

Dataset de Prueba

Filas:	885
Columnas:	37
Features:	36
ID:	1



3. Desarrollo de la Predicción

2.1 Transformación de Datos

Para preparar los datos antes del modelado, se aplicaron dos transformaciones principales: codificación de etiquetas y normalización de características.

El atributo objetivo fue convertido de categorías textuales a valores numéricos, siguiendo la siguiente correspondencia:

Esta transformación permite que el modelo interprete correctamente las clases durante el entrenamiento.

Clase Original	Código Numérico	Muestras	Porcentaje
Dropout	0	1,150	32,5%
Enrolled	1	642	18,1%
Graduate	2	1,747	49,4%

2.2 Resultados del Preprocesamiento

Se aplicó StandardScaler para escalar las variables, esto garantiza que todas las características tengan una escala comparable, mejorando el rendimiento del modelo.

$$z = \frac{(x - \mu)}{\sigma}$$

donde X es el valor de los datos, μ es la media y σ es la desviación estándar de la muestra

Conjunto	Media	Desviación	Mínimo	Máximo
X_train	0	1	-4.115.142	17.924.576
X_test	5.586	994.053	-4.115.142	17.924.576



3. Algoritmos y configuraciones

Se utilizó tres categorías principales de algoritmos para construir el Ensemble, sumando 11 modelos base con un total de 210 configuraciones evaluadas.

Tipo de Algoritmo	Modelos Base (11)	Configuraciones Generadas
1. Gradient Boosting	XGBoost, LightGBM, CatBoost, Gradient Boosting, HistGradientBoosting, AdaBoost	150
2. Forest y Bagging	Random Forest, Extra Trees, Bagging	40
3. Neuronales y Kernel	MLP Classifier (Red Neuronal), SVC (Máquinas de Soporte Vectorial)	20
TOTAL	11 Modelos Base	210 Configuraciones



3. Desarrollo de la Predicción

4. Configuraciones de cada modelo

El Gradient Boosting es la columna vertebral del Ensemble. Estos algoritmos trabajan secuencialmente, corrigiendo los errores del modelo anterior para reducir el sesgo. Son los modelos con mayor capacidad predictiva en datos tabulares y son esenciales para alcanzar el puntaje más alto. Su rol es asegurar la máxima precisión en la predicción.

XGBoost Classifier (40 configuraciones)

- n_estimators: [800, 1000, 1200, 1500, 1800]
- max_depth: [5, 6, 7, 8]
- learning_rate: {0.015, 0.02, 0.025, 0.03}
- reg_alpha/lambda: [0.2, 0.3, 0.5]

LightGBM Classifier (40 configuraciones)

- n_estimators [800, 1000, 1200, 1500, 1800]
- num_leaves [31, 40, 50, 60]
- learning_rate [0.015, 0.02, 0.025, 0.03]
- class_weight (cw) ['balanced', None]

CatBoost Classifier (30 configuraciones)

- iterations (n_est) [800, 1000, 1200, 1500, 1800]
- depth (max_d) [5, 6, 7, 8]
- learning_rate: [0.015, 0.02, 0.025, 0.03]

GradientBoosting (15 configuraciones)

- Classifier n_estimators [300, 500, 700]
- learning_rate (lr) [0.05, 0.1, 0.15]
- max_depth (max_d) [4, 5, 6]

HistGradientBoosting (15 configuraciones)

- learning_rate: [0.05, 0.1, 0.15]
- max_depth (max_d) [5, 10, 15, 20, 25]

AdaBoost Classifier (10 configuraciones)

- n_estimators [100, 200, 300, 400]
- learning_rate [0.5, 1.0, 1.5]



3. Desarrollo de la Predicción

4. Configuraciones de cada modelo

A diferencia del Boosting, estos modelos trabajan paralelamente (haciendo **bootstrapping** o **Bagging**). Entrenan múltiples árboles de forma independiente sobre subconjuntos de datos y features. Esto reduce la varianza y hace que las predicciones sean extremadamente estables y menos sensibles al ruido o a valores atípicos. Su rol es aportar robustez y fiabilidad al Ensemble.

RandomForest (20 configuraciones)

- Classifier n_estimators [300, 500, 700, 900]
- max_depth (max_d) [10, 15, 20, 25]
- class_weight (cw) ['balanced', None]

ExtraTrees Classifier (12 configuraciones)

- n_estimators [300, 500, 700]
- max_depth (max_d) [10, 15, 20, 25]

Bagging Classifier (8 configuraciones)

- n_estimators [50, 100, 150, 200]
- max_features (max_f) [0.8, 1.0]



3. Desarrollo de la Predicción

4. Configuraciones de cada modelo

Estos modelos abordan la clasificación desde una perspectiva diferente, buscando fronteras de decisión y patrones que no son fácilmente identificados por los árboles de decisión. La **Red Neuronal (MLP)** y las **SVC** son especialmente buenas para modelar relaciones no lineales y complejas entre las features. Su rol es garantizar una generalización más amplia, complementando las debilidades de los modelos basados en árboles.

MLP Neuronal (12 configuraciones)

- hidden_layer_sizes [(128, 64), (256, 128), (128, 64, 32), (256, 128, 64)]
- alpha [0.0001, 0.001, 0.01]

SVC (Kernel: rbf/poly) (8 configuraciones)

- C [0.5, 1.0, 2.0, 5.0]
- kernel ['rbf', 'poly']



3. Desarrollo de la Predicción

4.1 Distribución Final de Configuraciones

XGB: 40 modelos
(19.0%)

LGB: 40 modelos
(19.0%)

CAT: 30 modelos
(14.3%)

RF: 20 modelos
(9.5%)

GB: 15 modelos
(7.1%)

HGB: 15 modelos
(7.1%)

ET: 12 modelos
(5.7%)

MLP: 12 modelos
(5.7%)

ADA: 10 modelos
(4.8%)

SVC: 8 modelos
(3.8%)

BAG: 8 modelos
(3.8%)



3. Desarrollo de la Predicción

5. Configuración de Validación Cruzada

El uso de una validación cruzada utilizó 7-Fold Estratificada, el cual ofrece varias ventajas frente al enfoque tradicional.

En primer lugar, proporciona mayor robustez, ya que permite obtener una estimación más confiable del error de generalización. Además, al utilizar más datos de entrenamiento en cada iteración (85.7%) los modelos pueden aprender con un conjunto más amplio y representativo.

Este esquema también favorece una validación más estricta, reduciendo la varianza en los puntajes Out-Of-Fold y brindando resultados más estables.

Finalmente, garantiza un mejor aprovechamiento de los datos, ya que cada muestra del conjunto se valida exactamente una vez, maximizando así la eficiencia del proceso de evaluación.

Para evaluar los modelos se utilizó una validación cruzada StratifiedKFold con las siguientes características:

```
Tipo: StratifiedKFold  
n_splits: 7  
shuffle: True  
random_state: 42
```

En total, se realizaron 1,470 evaluaciones (210 modelos $\times$ 7 folds).



5.1 Estadísticas de OOF Scores

Se evaluaron múltiples modelos con diferentes configuraciones, a los cuales se les proporcionaron los mismos conjuntos de entrenamiento y prueba. Posteriormente, sus resultados **OOF** (Out-Of-Fold) predice en datos no vistos durante validación cruzada, evalúa rendimiento real, evita sobreajuste y selecciona modelos confiables para ensemble.

Rank	Modelo	OOF Score	Desviación
1	XGB #11	7.799,00	± 0.0113
2	XGB #6	7.793,00	± 0.0117
3	XGB #26	7.788,00	± 0.0109
4	LGB #50	7.788,00	± 0.0123
5	ET #138	7.785,00	± 0.0159
6	ET #142	7.782,00	± 0.0147
7	XGB #3	7.779,00	± 0.0077
8	XGB #17	7.779,00	± 0.0091
9	XGB #10	7.776,00	± 0.0111
10	XGB #27	7.776,00	± 0.0107



5.2 Proceso de Reentrenamiento del TOP 50

1) Hiperparámetros fijos

Se mantienen los valores (`learning_rate`, `max_depth`, `n_estimators`, etc.) que dieron el mejor OOF para cada uno de los 50 modelos.

Ejemplo: si XGBoost con `max_depth=7` y `lr=0.02` fue el segundo mejor, se reutiliza esa misma configuración para el reentrenamiento.

2) Entrenamiento con todos los datos

Cada modelo se entrena desde cero con su configuración ganadora, ahora usando el 100% de `X_train` e `y_train` (no solo la parte usada en los folds de 7-Fold CV).

3) Predicciones finales de probabilidad

Tras el reentrenamiento, los 50 modelos generan probabilidades `P_modelo` sobre `X_test`. Estas probabilidades serán la base de la Etapa 4 de resultados: Suma Ponderada (Ensemble Dual).



5.3 Cálculo de Pesos Optimizados

Tres Estrategias de Pesos Implementadas:

1. POW4

Énfasis moderado en mejores modelos
mediante función potencia

Rango pesos: [0.019820, 0.020345]

Desviación: 0.000127

$$\text{weight}_i = \frac{\text{score}_i^4}{\sum_{j=1}^n \text{score}_j^4}$$

2. EXP20

Énfasis fuerte mediante función exponencial

Rango pesos: [0.019305, 0.021373]

Desviación: 0.000497

$$\text{weight}_i = \frac{\exp(\text{score}_i \times 20)}{\sum_{j=1}^n \exp(\text{score}_j \times 20)}$$

3. MIX 50-50

Balance óptimo: estabilidad de POW4 + sensibilidad de
EXP20

Rango pesos: [0.019563, 0.020859]

Desviación: 0.000312

$$\text{weight}_i = 0.5 \times \left(\frac{\text{score}_i^4}{\sum_{j=1}^n \text{score}_j^4} \right) + 0.5 \times \left(\frac{\exp(\text{score}_i \times 20)}{\sum_{j=1}^n \exp(\text{score}_j \times 20)} \right)$$



3. Desarrollo de la Predicción

5.3 Cálculo de Pesos Optimizados

Comparativa de Estrategias				
Estrategia	Peso Mínimo	Peso Máximo	Desviación	Ventaja
POW4	0.019820	0.020345	0.000127	Más estable
EXP20	0.019305	0.021373	0.000497	Más sensible
MIX 50-50	0.019563	0.020859	0.000312	Equilibrado

Rank	Tipo	Mix	Pow4	Exp	OOF
1	XGB #11	209	203	214	7.799
2	XGB #6	207	203	211	7.793
3	XGB #26	206	202	209	7.788
4	LGB #50	206	202	209	7.788
5	ET #138	205	202	208	7.785
6	ET #142	204	202	207	7.782
7	XGB #3	203	201	205	7.779
8	XGB #17	203	201	205	7.779
9	XGB #10	203	201	204	7.776
10	XGB #27	203	201	204	7.776



Preparación del Archivo Final

1. Generación de Probabilidades

Los 50 modelos reentrenados generan sus matrices de probabilidad individuales para el conjunto de prueba (X_{test}). 50 matrices de 885 x 3 probabilidades.

2. Aplicar Ponderación MIX

Cada matriz P_i se multiplica por el Peso MIX_i calculado previamente.

3. Suma de Probabilidades

Se suman las 50 matrices de probabilidades ponderadas para obtener una única Matriz Combinada de Scores: $P_{Combinada}$. 1 matriz de 885 x 3 scores combinados.

4. Normalización Crucial

Cada fila de la Matriz Combinada se divide por la suma de sus propios elementos, asegurando que las probabilidades de cada fila sumen 1.0.

5. Decisión de la Clase (Argmax)

Se aplica el argmax a lo largo de cada fila para seleccionar la columna (clase) con la probabilidad más alta. (índices 0, 1, 2).



4. Resultados

Preparación del Archivo Final

6. Decodificación

El índice numérico (0, 1, 2) obtenido del argmax se convierte de nuevo a las etiquetas de texto originales (Dropout, Enrolled, Graduate).

7. Codificación One-Hot (0s y 1s)

La etiqueta final se transforma en un formato de tres columnas donde se pone un 1 en la columna de la clase predicha y 0 en las otras dos.

submission.csv

id	Dropout	Enrolled	Graduate
0	0	0	1,0
1	0	0	1,0
2	0	1,0	0

Distribución Final de Predicciones

Observación: Las tres estrategias produjeron distribuciones idénticas, validando la robustez del ensemble.

251

Dropout



28.4%

Prob media: 0.304 ± 0.358

110

Enrolled



12.4%

Prob media: 0.174 ± 0.202

524

Graduate



59.2%

Prob media: 0.522 ± 0.393



4. Resultados

Puntuación Score

Leaderboard Público (70% de los Datos de Prueba)

Ofrece una retroalimentación inmediata sobre qué tan bien se está desempeñando su modelo.

score: 0.8174

Leaderboard Privado (30% de los Datos de Prueba)

Resultado final del leaderboard, si un modelo se sobreajustó al 70% público, su rendimiento caerá significativamente en este 30% privado, revelando una mala generalización.

score: 0.7706

La diferencia entre sus dos scores indica que el modelo tuvo un grado de sobreajuste (overfitting) a los datos que eran visibles en el leaderboard público.



5. Conclusiones

El modelo mostró un rendimiento notable en los patrones visibles; sin embargo, no logró generalizar de manera efectiva a los patrones nuevos y no observados en el conjunto privado. Esto indica que, aunque el uso de la validación cruzada de 7 pliegues y las estrategias de ensamblaje son prometedoras, la configuración final, como la intensidad de los pesos EXP20 o el número de iteraciones de los potenciadores, resultó ser demasiado específica para la distribución de los datos públicos.



6. Bibliografía

- [1] GeeksforGeeks. (2025, August 28). *A comprehensive guide to ensemble learning*.
<https://www.geeksforgeeks.org/machine-learning/a-comprehensive-guide-to-ensemble-learning>
- [2] Singh, A. (2025, April 4). *A comprehensive guide to ensemble learning (with Python codes)*. Analytics Vidhya.
<https://www.analyticsvidhya.com/blog/2018/06/comprehensive-guide-for-ensemble-models>
- [3] Van Otten, N. (2023, August 9). *Top 6 most powerful ensemble learning techniques explained & algorithms that implement them*. Spot Intelligence. <https://spotintelligence.com/2023/08/09/ensemble-learning>
- [4] Brownlee, J. (2021, May 8). *How to develop a weighted average ensemble with Python*. Machine Learning Mastery.
<https://machinelearningmastery.com/weighted-average-ensemble-with-python>
- [5] Douglass, M. J. J. (2020). Book Review: *Hands-on Machine Learning with Scikit-Learn, Keras, and Tensorflow, 2nd edition by Aurélien Géron*. Physical and Engineering Sciences in Medicine, 43, 1135–1136. <https://doi.org/10.1007/s13246-020-00913-z>
-



Universidad
Finis Terrae®